

```
=====
GCI – GYKHAMINE CONCEPT INVESTIGATION
COURS EXHAUSTIF : WEB SCRAPING AVEC PYTHON
requests · httpx · Scrapy · BeautifulSoup · lxml · parsel · selenium
=====
```

```
Auteur  : GCI – Département Formation & Intelligence Analytique
Version : 1.0 – Édition Complète
Format  : Bible Professionnelle
Langue  : Français
=====
```

TABLE DES MATIÈRES

```
PARTIE I   – FONDEMENTS DU WEB SCRAPING
PARTIE II  – REQUÊTES HTTP : REQUESTS
PARTIE III – REQUÊTES HTTP MODERNES : HTTPX (SYNC & ASYNC)
PARTIE IV  – PARSING HTML : BEAUTIFULSOUP4
PARTIE V   – PARSING AVANCÉ : LXML ET PARSEL (XPath / CSS)
PARTIE VI  – SCRAPY : FRAMEWORK COMPLET DE CRAWLING
PARTIE VII – SCRAPY AVANCÉ : ITEMS, LOADERS, PIPELINES
PARTIE VIII – GESTION DES URLS : URLLIB, W3LIB, TLDEXTRACT, HYPERLINK
PARTIE IX  – ENCODAGE ET NORMALISATION : CHARSET_NORMALIZER, WEBENCODINGS
PARTIE X   – SCRAPING ASYNCHRONE : AIOHTTP + ASYNCIO
PARTIE XI  – PROXIES ET ANONYMAT : SOCKS, SOCKSHANDLER, URLLIB3
PARTIE XII – SCRAPING DE SITES DYNAMIQUES : SELENIUM
PARTIE XIII – NETTOYAGE ET POST-TRAITEMENT : BLEACH, W3LIB, RE, REGEX
PARTIE XIV – STOCKAGE DES DONNÉES SCRAPÉES : JSON, CSV, PANDAS, SQLITE
PARTIE XV  – BONNES PRATIQUES, ÉTHIQUE ET ROBUSTESSE
PARTIE XVI – PROJETS COMPLETS DE A À Z
```

PARTIE I – FONDEMENTS DU WEB SCRAPING

1.1 QU'EST-CE QUE LE WEB SCRAPING ?

Le web scraping est le processus automatisé d'extraction de données depuis des pages web. On distingue plusieurs niveaux :

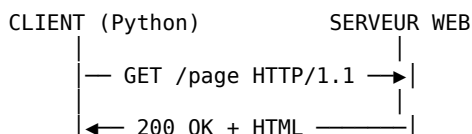
```
CRAWLING   → Parcourir automatiquement des liens pour découvrir des pages
SCRAPING    → Extraire des données structurées depuis des pages HTML
PARSING     → Analyser le HTML/XML pour en extraire les éléments souhaités
PIPELINE    → Nettoyer, transformer et stocker les données extraites
```

QUAND UTILISER QUEL OUTIL :

BESOIN	→ OUTIL RECOMMANDÉ
Requête HTTP simple	→ requests
Requêtes async ou HTTP/2	→ httpx
Parser du HTML simple	→ BeautifulSoup4
XPath / CSS haute performance	→ lxml, parsel
Crawler multi-pages / prod	→ Scrapy
Site dynamique (JavaScript)	→ Selenium
Scraping async massif	→ aiohttp + asyncio
Nettoyage du HTML extrait	→ bleach, w3lib

1.2 ANATOMIE D'UNE PAGE WEB – CE QUE LE SCRAPER VOIT

Quand on fait une requête HTTP GET vers une URL :



La réponse contient :

- STATUS CODE : 200 (OK), 404 (Not Found), 403 (Forbidden), 429 (Rate limit)

- HEADERS : Content-Type, Set-Cookie, Content-Encoding, Server...
- BODY : le HTML brut de la page

Structure HTML type :

```
<html>
  <head>
    <title>Titre</title>
  </head>
  <body>
    <div class="produit" id="p001">
      <h2>Nom du produit</h2>
      <span class="prix">29,90 €</span>
      <a href="/produit/p001">Voir détail</a>
    </div>
  </body>
</html>
```

OUTILS D'IDENTIFICATION DES SÉLECTEURS :

- Dans Chrome/Firefox : F12 → Inspecter l'élément → Clic droit → "Copy selector" ou "Copy XPath"

1.3 INSTALLATION DES MODULES

```
# Installation complète pour ce cours
pip install requests httpx scrapy beautifulsoup4 lxml parsel \
    selenium aiohttp w3lib tldextract charset-normalizer \
    urllib3 PySocks bleach pandas tqdm
```

PARTIE II – REQUÊTES HTTP : REQUESTS

2.1 PRÉSENTATION DU MODULE REQUESTS

requests est le client HTTP le plus utilisé en Python. Il encapsule urllib3 et offre une API simple et lisible.

Modules associés du fichier de documentation :

- requests : client principal
- requests_file : adapter pour lire des fichiers locaux via file://
- requests_oauthlib: OAuth 1 & 2 pour requests
- responses : mock de requêtes pour les tests
- urllib3 : transport bas niveau (pooling, retry, SSL)
- certifi : certificats CA racines (SSL)

2.2 REQUÊTES DE BASE

```
import requests

# GET simple
response = requests.get("https://example.com")
print(response.status_code)    # 200
print(response.headers)        # dictionnaire des headers
print(response.text)           # HTML en str
print(response.content)        # HTML en bytes
print(response.url)            # URL finale (après redirections)

# GET avec paramètres d'URL (?q=python&page=2)
params = {"q": "python scraping", "page": 2}
response = requests.get("https://example.com/search", params=params)
print(response.url)            # https://example.com/search?q=python+scraping&page=2

# POST avec données de formulaire
data = {"username": "admin", "password": "secret"}
response = requests.post("https://example.com/login", data=data)

# POST avec JSON
payload = {"titre": "Article", "contenu": "Texte..."}
response = requests.post("https://api.example.com/articles",
```

```
json=payload)
```

2.3 HEADERS ET USER-AGENT

La plupart des sites bloquent les requêtes sans User-Agent valide.
Toujours simuler un navigateur réel.

```
headers = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/120.0.0.0 Safari/537.36"
    ),
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    "Accept-Language": "fr-FR,fr;q=0.9,en-US;q=0.8",
    "Accept-Encoding": "gzip, deflate, br",
    "Connection": "keep-alive",
    "Referer": "https://www.google.com/"
}

response = requests.get("https://example.com", headers=headers)
```

2.4 SESSIONS : MAINTENIR LES COOKIES

Une Session persiste les cookies, headers et paramètres entre les requêtes.
Indispensable pour naviguer sur un site après connexion.

```
import requests

session = requests.Session()

# Définir des headers par défaut pour toute la session
session.headers.update({
    "User-Agent": "Mozilla/5.0 ...",
    "Accept-Language": "fr-FR"
})

# Connexion (les cookies de session sont stockés automatiquement)
login_data = {"email": "user@example.com", "password": "pass123"}
session.post("https://example.com/login", data=login_data)

# Pages suivantes utilisent les cookies de connexion
page_profil = session.get("https://example.com/profil")
page_commandes = session.get("https://example.com/commandes")

# Accéder aux cookies
print(dict(session.cookies))

# Fermer proprement
session.close()

# Utiliser comme context manager (recommandé)
with requests.Session() as session:
    session.get("https://example.com")
```

2.5 GESTION DES ERREURS ET RETRY

```
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry
import time

def creer_session_robuste():
    """Session avec retry automatique sur erreurs réseau."""
    session = requests.Session()

    retry_strategy = Retry(
        total=3,                      # 3 tentatives au total
        backoff_factor=1,             # délai : 1s, 2s, 4s
    )
    session.mount("https://", HTTPAdapter(max_retries=retry_strategy))

    return session
```

```

        status_forcelist=[429, 500, 502, 503, 504], # codes à réessayer
        allowed_methods=["GET", "POST"]
    )

    adapter = HTTPAdapter(max_retries=retry_strategy)
    session.mount("https://", adapter)
    session.mount("http://", adapter)
    return session

def get_safe(url, session=None, timeout=10):
    """GET avec gestion complète des erreurs."""
    if session is None:
        session = requests.Session()
    try:
        response = session.get(url, timeout=timeout)
        response.raise_for_status() # lève une exception si 4xx/5xx
        return response
    except requests.exceptions.HTTPError as e:
        print(f"Erreur HTTP {e.response.status_code} : {url}")
    except requests.exceptions.ConnectionError:
        print(f"Erreur de connexion : {url}")
    except requests.exceptions.Timeout:
        print(f"Timeout : {url}")
    except requests.exceptions.RequestException as e:
        print(f"Erreur inattendue : {e}")
    return None

# Utilisation
session = creer_session_robuste()
response = get_safe("https://example.com/page", session=session)
if response:
    print(response.text[:500])

```

2.6 TÉLÉCHARGEMENT DE FICHIERS ET STREAMING

```

import requests
from tqdm import tqdm # barre de progression

def telecharger_fichier(url, chemin_local):
    """Télécharge un fichier avec barre de progression."""
    with requests.get(url, stream=True, timeout=30) as response:
        response.raise_for_status()
        taille_totale = int(response.headers.get("content-length", 0))

        with open(chemin_local, "wb") as f, \
            tqdm(total=taille_totale, unit="B", unit_scale=True) as pbar:
            for chunk in response.iter_content(chunk_size=8192):
                f.write(chunk)
                pbar.update(len(chunk))

telecharger_fichier("https://example.com/dataset.csv", "data/dataset.csv")

```

2.7 REQUESTS_FILE ET REQUESTS_OAUTHLIB

```

# requests_file : lire des fichiers locaux avec requests
import requests
from requests_file import FileAdapter

session = requests.Session()
session.mount("file://", FileAdapter())
response = session.get("file:///home/user/page_locale.html")
print(response.text[:200])

# requests_oauthlib : OAuth2 pour API authentifiées (ex: Twitter, GitHub)
from requests_oauthlib import OAuth2Session

client_id = "VOTRE_CLIENT_ID"
redirect_uri = "https://votre-app.com/callback"
oauth = OAuth2Session(client_id, redirect_uri=redirect_uri,
                      scope=["read:user", "public_repo"])

```

```

authorization_url, state = oauth.authorization_url(
    "https://github.com/login/oauth/authorize"
)
print(f"Alliez sur : {authorization_url}")

```

PARTIE III – REQUÊTES HTTP MODERNES : HTTPX (SYNC & ASYNC)

3.1 PRÉSENTATION DE HTTPX

httpx est le successeur moderne de requests. Il offre :

- API quasi identique à requests (migration facile)
- Support natif de HTTP/2 (plus rapide sur certains sites)
- Client asynchrone intégré (asyncio)
- Timeouts par phase (connexion, lecture, écriture)
- Meilleure gestion des redirections

Modules associés :

- httpx : client principal
- httpcore : transport bas niveau utilisé par httpx

3.2 UTILISATION SYNCHRONE

```

import httpx

# GET simple (syntaxe identique à requests)
response = httpx.get("https://example.com")
print(response.status_code)
print(response.text)

# Client réutilisable avec configuration persistante
with httpx.Client(
    headers={"User-Agent": "Mozilla/5.0 ..."},
    timeout=httpx.Timeout(10.0, connect=5.0),
    follow_redirects=True,
    http2=True # activer HTTP/2 (nécessite pip install httpx[http2])
) as client:

    # GET avec paramètres
    r = client.get("https://httpbin.org/get", params={"page": 1})
    print(r.json())

    # POST JSON
    r = client.post("https://httpbin.org/post",
                    json={"cle": "valeur"})
    print(r.json())

```

3.3 UTILISATION ASYNCHRONE – LE VRAI ATOUT DE HTTPX

Le client asynchrone permet de lancer des dizaines de requêtes en parallèle sans threading, en utilisant asyncio.

```

import httpx
import asyncio

# Requête unique asynchrone
async def fetch_page(url):
    async with httpx.AsyncClient(follow_redirects=True) as client:
        response = await client.get(url, timeout=10)
        return response.text

# Scraping parallèle de plusieurs URLs
async def scraper_plusieurs_urls(urls):
    async with httpx.AsyncClient(
        headers={"User-Agent": "Mozilla/5.0 ..."},
        timeout=15.0,
        follow_redirects=True
    ) as client:

```

```

taches = [client.get(url) for url in urls]
resultats = await asyncio.gather(*taches, return_exceptions=True)

pages = {}
for url, result in zip(urls, resultats):
    if isinstance(result, Exception):
        print(f"Erreur pour {url} : {result}")
    elif result.status_code == 200:
        pages[url] = result.text
    else:
        print(f"Status {result.status_code} pour {url}")
return pages

# Exécution
urls = [
    "https://example.com/page/1",
    "https://example.com/page/2",
    "https://example.com/page/3",
]
pages = asyncio.run(scraper_plusieurs_urls(urls))
print(f"{len(pages)} pages récupérées.")

```

3.4 HTTPX AVEC LIMITES DE CONCURRENCE

Lancer 1000 requêtes simultanément crashe les serveurs et fait bloquer votre IP.
Utiliser un Semaphore pour limiter la concurrence.

```

import httpx
import asyncio

async def fetch_with_semaphore(client, url, semaphore):
    async with semaphore:
        try:
            r = await client.get(url, timeout=10)
            await asyncio.sleep(0.5) # délai poli entre requêtes
            return url, r.text
        except Exception as e:
            return url, None

async def scraper_avec_limite(urls, max_concurrent=5):
    semaphore = asyncio.Semaphore(max_concurrent)
    async with httpx.AsyncClient(follow_redirects=True) as client:
        taches = [
            fetch_with_semaphore(client, url, semaphore)
            for url in urls
        ]
        resultats = await asyncio.gather(*taches)
    return dict(resultats)

```

PARTIE IV – PARSING HTML : BEAUTIFULSOUP4

4.1 PRÉSENTATION DE BS4

BeautifulSoup4 (bs4) transforme le HTML brut en un arbre d'objets Python navigable. Il est tolérant aux HTML malformés.

Modules associés :

- bs4 : bibliothèque principale
- soupsieve : moteur de sélecteurs CSS pour BS4 (select, select_one)
- lxml : parseur recommandé (rapide, strict)
- html5lib : parseur alternatif (très conforme, plus lent)
- html : module stdlib pour escape/unescape

CHOIX DU PARSEUR :

```

BeautifulSoup(html, "html.parser") # stdlib Python, lent, tolérant
BeautifulSoup(html, "lxml")        # C, rapide, recommandé
BeautifulSoup(html, "html5lib")     # très conforme HTML5, lent

```

4.2 NAVIGATION DANS L'ARBRE HTML

```
import requests
from bs4 import BeautifulSoup

html = requests.get("https://quotes.toscrape.com").text
soup = BeautifulSoup(html, "lxml")

# ACCÈS PAR TAG
print(soup.title)           # <title>Quotes to Scrape</title>
print(soup.title.text)      # Quotes to Scrape
print(soup.h1)              # premier <h1>
print(soup.h1.string)        # texte direct du h1

# ACCÈS AUX ATTRIBUTS
lien = soup.find("a")
print(lien["href"])          # valeur de href
print(lien.get("class"))      # liste des classes CSS

# NAVIGATION DANS L'ARBRE
div = soup.find("div", class_="quote")
print(div.parent.name)        # nom du parent
print(div.children)           # enfants directs (générateur)
print(div.descendants)          # tous les descendants
print(div.next_sibling)       # sibling suivant
print(div.previous_sibling)    # sibling précédent
```

4.3 FIND ET FIND_ALL : RECHERCHE D'ÉLÉMENTS

```
from bs4 import BeautifulSoup
import requests

soup = BeautifulSoup(requests.get("https://quotes.toscrape.com").text, "lxml")

# find() → premier élément correspondant
premier_auteur = soup.find("small", class_="author")
print(premier_auteur.text)    # "Albert Einstein"

# find_all() → tous les éléments correspondants (liste)
tous_les_auteurs = soup.find_all("small", class_="author")
for auteur in tous_les_auteurs:
    print(auteur.text)

# find_all avec plusieurs classes
elements = soup.find_all("div", class_=["quote", "item"])

# find_all avec attribut ID
element = soup.find("div", id="main-content")

# find_all avec attribut quelconque
liens_externes = soup.find_all("a", href=True)

# find_all avec expression régulière
import re
titres = soup.find_all("h1", string=re.compile("Python"))

# find_all avec fonction lambda
grands_textes = soup.find_all(
    lambda tag: tag.name == "p" and len(tag.text) > 200
)

# Limiter le nombre de résultats
premiers_cinq = soup.find_all("li", limit=5)
```

4.4 SÉLECTEURS CSS AVEC SELECT (SOUPSIEVE)

La méthode `select()` utilise `soupsieve` pour interroger le HTML avec des sélecteurs CSS, souvent plus lisibles que `find/find_all`.

```

# Sélecteur simple
citations = soup.select("div.quote")          # <div class="quote">
tags = soup.select("div.quote span.tag")      # span dans div
auteurs = soup.select("small.author")

# select_one() → équivalent CSS de find()
titre = soup.select_one("h1.title")

# Sélecteurs avancés
soup.select("a[href]")                       # liens avec attribut href
soup.select("a[href^='https']")              # href commence par https
soup.select("a[href$='.pdf']")                # href finit par .pdf
soup.select("a[href*='example']")            # href contient example
soup.select("div > p")                        # p enfant direct de div
soup.select("div + p")                       # p immédiatement après div
soup.select("h2 ~ p")                        # p sibling de h2
soup.select("li:nth-child(2)")                # 2ème li enfant
soup.select("li:first-child")                # 1er enfant
soup.select("li:last-child")                 # dernier enfant
soup.select("p:not(.intro)")                 # p sans classe intro

```

4.5 EXTRACTION DE DONNÉES – EXEMPLE COMPLET

```

import requests
from bs4 import BeautifulSoup
import pandas as pd

def scraper_citations():
    """Scrape toutes les citations de quotes.toscrape.com."""
    base_url = "https://quotes.toscrape.com"
    page = 1
    toutes_citations = []

    while True:
        url = f"{base_url}/page/{page}/"
        response = requests.get(url, timeout=10)

        if response.status_code != 200:
            break

        soup = BeautifulSoup(response.text, "lxml")
        citations = soup.select("div.quote")

        if not citations:
            break

        for citation in citations:
            texte = citation.select_one("span.text").text.strip()
            auteur = citation.select_one("small.author").text.strip()
            tags = [t.text for t in citation.select("a.tag")]

            toutes_citations.append({
                "texte": texte,
                "auteur": auteur,
                "tags": ", ".join(tags),
                "nb_tags": len(tags)
            })

        # Vérifier s'il y a une page suivante
        suivant = soup.select_one("li.next a")
        if not suivant:
            break
        page += 1
        print(f"Page {page} extraite ({len(toutes_citations)} citations)")

    return pd.DataFrame(toutes_citations)

df = scraper_citations()
print(df.head())
df.to_csv("citations.csv", index=False, encoding="utf-8")

```

5.1 LXML : LE PARSEUR LE PLUS RAPIDE

lxml est une bibliothèque C (libxml2/libxslt) exposée en Python. Elle est 5 à 10x plus rapide que html.parser pour les gros documents.

```
from lxml import html, etree
import requests

response = requests.get("https://example.com")
# Parser le HTML directement
tree = html.fromstring(response.content)

# XPath : langage de requête pour les arbres XML/HTML
titres = tree.xpath("//h2[@class='titre']/text()")
liens = tree.xpath("//a/@href")
prix = tree.xpath("//span[@class='prix']/text()")

# CSS selector avec lxml + cssselect
# (nécessite pip install cssselect)
divs = tree.cssselect("div.produit")
for div in divs:
    nom = div.cssselect("h3")[0].text_content()
    print(nom)
```

5.2 XPATH : SYNTAXE COMPLÈTE

XPath est un langage de navigation dans les arbres XML/HTML.
Syntaxe de base :

//tag	→ tous les <tag> dans le document
/html/body/div	→ chemin absolu
//div[@class='x']	→ div avec classe x
//div[@id='main']	→ div avec id main
//a/@href	→ valeur de l'attribut href de tous les <a>
//h2/text()	→ texte direct des <h2>
//div//p	→ tous les <p> descendants d'un <div>
//div/p	→ <p> enfants directs de <div>
((div)[1])	→ premier <div> du document
//li[last()]	→ dernier
//li[position()>2]	→ après le 2ème
//div[contains(@class, 'produit')]	→ class contient 'produit'
//p[starts-with(text(), 'Python')]	→ texte commence par Python
//p[string-length(text()) > 100]	→ texte > 100 caractères
//tr[td='Valeur']	→ ligne de tableau contenant 'Valeur'

EXEMPLE PRATIQUE :

```
from lxml import html
import requests

page = html.fromstring(requests.get("https://books.toscrape.com").content)

# Extraire tous les livres
livres = page.xpath("//article[@class='product_pod']")
for livre in livres:
    titre = livre.xpath("./h3/a/@title")[0]
    prix = livre.xpath("./p[@class='price_color']/text()")[0]
    dispo = livre.xpath("./p[contains(@class,'availability')]/text()")
    dispo = dispo[0].strip() if dispo else "Inconnu"
    print(f"{titre} | {prix} | {dispo}")
```

5.3 PARSEL : XPATH + CSS STYLE SCRAPY SANS SCRAPY

parsel est le moteur d'extraction de Scrapy, utilisable de manière indépendante. Il combine XPath et CSS dans une API fluide et chainable.

```

import parsel
import requests

html_content = requests.get("https://quotes.toscrape.com").text
selector = parsel.Selector(text=html_content)

# CSS selector
auteurs = selector.css("small.author::text").getall()
print(auteurs) # ['Albert Einstein', 'J.K. Rowling', ...]

# XPath
textes = selector.xpath("//span[@class='text']/text()").getall()

# get() = premier résultat (équivalent find())
titre = selector.css("h1::text").get()

# Chaining : sélectionner dans un sous-arbre
for citation in selector.css("div.quote"):
    texte = citation.css("span.text::text").get()
    auteur = citation.css("small.author::text").get()
    tags = citation.css("a.tag::text").getall()
    print(f"{auteur}: {texte[:50]}... [{', '.join(tags)}]")

# Attributs avec CSS
liens = selector.css("a::attr(href)").getall()
images = selector.css("img::attr(src)").getall()

# Attributs avec XPath
hrefs = selector.xpath("//a/@href").getall()

# re() : extraction par expression régulière directement
prix = selector.css("p.price::text").re(r"\d+\.\d+")

```

PARTIE VI – SCRAPY : FRAMEWORK COMPLET DE CRAWLING

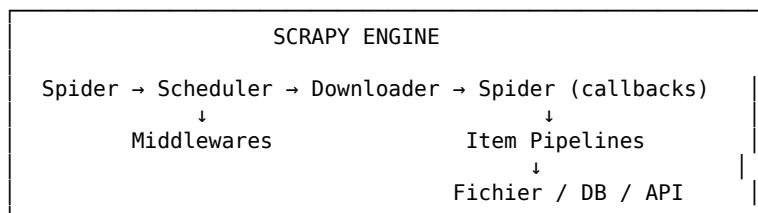
6.1 PRÉSENTATION ET ARCHITECTURE DE SCRAPY

Scrapy est un framework de scraping complet, asynchrone, basé sur Twisted. Il gère automatiquement : files de requêtes, déduplication d'URLs, throttling, middlewares, pipelines de données, exports multi-formats.

Modules associés du fichier de documentation :

- scrapy : framework principal
- parsel : extraction XPath/CSS dans les spiders
- itemadapter : interface uniforme pour les items Scrapy
- itemloaders : chargement et nettoyage des items
- w3lib : utilitaires URL, HTML, encodage
- queuelib : files de priorité pour la scheduler Scrapy
- twisted : moteur réseau asynchrone sous-jacent
- service_identity : vérification TLS pour Twisted
- cssselect : sélecteurs CSS vers XPath

ARCHITECTURE SCRAPY :



6.2 CRÉER UN PROJET SCRAPY

```

# En ligne de commande
scrapy startproject mon_scraper
cd mon_scraper

```

```
scrapy genspider citations quotes.toscrape.com
```

```
# Structure créée :
# mon_scraper/
# └─ scrapy.cfg
# └─ mon_scraper/
#     └─ __init__.py
#         └─ items.py
#             └─ middlewares.py
#                 └─ pipelines.py
#                     └─ settings.py
#                         └─ spiders/
#                             └─ citations.py
```

6.3 SPIDER DE BASE

```
# spiders/citations.py
import scrapy

class CitationsSpider(scrapy.Spider):
    name = "citations"
    allowed_domains = ["quotes.toscrape.com"]
    start_urls = ["https://quotes.toscrape.com/"]

    custom_settings = {
        "DOWNLOAD_DELAY": 1,          # délai entre requêtes (secondes)
        "RANDOMIZE_DOWNLOAD_DELAY": True,
        "CONCURRENT_REQUESTS": 4,
        "ROBOTSTXT_OBEY": True,
    }

    def parse(self, response):
        """Callback principal : traite chaque page de liste."""

        # Extraction avec parsel (intégré dans response)
        for citation in response.css("div.quote"):
            yield {
                "texte": citation.css("span.text::text").get(),
                "auteur": citation.css("small.author::text").get(),
                "tags": citation.css("a.tag::text").getall(),
                "url": response.url,
            }

        # Suivre le lien "Suivant" → crawl multi-pages
        lien_suivant = response.css("li.next a::attr(href)").get()
        if lien_suivant:
            yield response.follow(lien_suivant, callback=self.parse)

# Lancer le spider
# scrapy crawl citations -o resultats.json
# scrapy crawl citations -o resultats.csv
# scrapy crawl citations -o resultats.jsonl
```

6.4 SPIDER AVEC PAGE DE DÉTAIL

```
import scrapy

class ProduitsSpider(scrapy.Spider):
    name = "produits"
    start_urls = ["https://books.toscrape.com/"]

    def parse(self, response):
        """Page de liste : extraire les liens vers les détails."""
        for lien in response.css("article.product_pod h3 a::attr(href)":
            yield response.follow(lien, callback=self.parse_detail)

        # Pagination
        suivant = response.css("li.next a::attr(href)").get()
        if suivant:
            yield response.follow(suivant, callback=self.parse)
```

```

def parse_detail(self, response):
    """Page de détail d'un livre."""
    def get_info(label):
        """Extraire une ligne du tableau d'infos."""
        return response.xpath(
            f"//th[text()=' {label}']/following-sibling::td/text()"
        ).get()

    yield {
        "titre": response.css("h1::text").get(),
        "prix": response.css("p.price_color::text").get(),
        "disponibilite": response.css(
            "p.availability::text").re_first(r"\w+"),
        "note": response.css("p.star-rating::attr(class)").re_first(
            r"star-rating (\w+)"),
        "upc": get_info("UPC"),
        "nb_avis": get_info("Number of reviews"),
        "description": response.css(
            "#product_description + p::text").get(),
        "url": response.url,
    }

```

6.5 CONFIGURATION SCRAPY (settings.py)

```

# settings.py – paramètres importants

BOT_NAME = "mon_scraper"

# Respecter robots.txt (recommandé)
ROBOTSTXT_OBEY = True

# Délai entre requêtes (poli)
DOWNLOAD_DELAY = 1.0
RANDOMIZE_DOWNLOAD_DELAY = True

# Concurrency
CONCURRENT_REQUESTS = 8
CONCURRENT_REQUESTS_PER_DOMAIN = 4

# Timeout
DOWNLOAD_TIMEOUT = 30

# User-Agent
USER_AGENT = (
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
    "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0"
)

# Activer les cookies
COOKIES_ENABLED = True

# Activer la compression automatique
COMPRESSION_ENABLED = True

# Pipelines actifs (numéro = priorité, plus bas = priorité haute)
ITEM_PIPELINES = {
    "mon_scraper.pipelines.NettoyagePipeline": 100,
    "mon_scraper.pipelines.StockagePipeline": 200,
}

# Middlewares
DOWNLOADER_MIDDLEWARES = {
    "scrapy.downloadermiddlewares.useragent.UserAgentMiddleware": None,
    "mon_scraper.middlewares.RotationUserAgentMiddleware": 400,
}

# Cache (évite de re-télécharger pendant le développement)
HTTPCACHE_ENABLED = True
HTTPCACHE_EXPIRATION_SECS = 3600
HTTPCACHE_DIR = ".scrapy/httpcache"

```

PARTIE VII – SCRAPY AVANCÉ : ITEMS, LOADERS, PIPELINES

7.1 ITEMS : DÉFINIR LA STRUCTURE DES DONNÉES

```
# items.py
import scrapy

class ProduitItem(scrapy.Item):
    titre = scrapy.Field()
    prix = scrapy.Field()
    prix_numerique = scrapy.Field()
    disponibilite = scrapy.Field()
    note = scrapy.Field()
    description = scrapy.Field()
    url = scrapy.Field()
    date_scrape = scrapy.Field()
```

7.2 ITEMLOADERS : NETTOYAGE AUTOMATIQUE À L'EXTRACTION

ItemLoaders appliquent automatiquement des fonctions de nettoyage sur les données extraites.

```
# spiders/produits_loader.py
import scrapy
from itemloaders import ItemLoader
from itemloaders.processors import TakeFirst, MapCompose, Join
from w3lib.html import remove_tags
import re
from mon_scraper.items import ProduitItem

def nettoyer_prix(valeur):
    """Convertit '£29.90' en 29.90"""
    return float(re.sub(r"^\d.", "", valeur))

def nettoyer_texte(valeur):
    """Supprime les tags HTML et normalise les espaces."""
    return remove_tags(valeur).strip()

class ProduitLoader(ItemLoader):
    default_output_processor = TakeFirst() # garder seulement le 1er résultat

    # Processeurs d'entrée : appliqués à l'extraction
    description_in = MapCompose(nettoyer_texte)
    prix_numerique_in = MapCompose(nettoyer_prix)

    # Processeur de sortie : appliqué à la liste complète
    tags_out = Join(", ") # liste → "tag1, tag2, tag3"

# Utilisation dans le spider
class ProduitsSpider(scrapy.Spider):
    name = "produits_propres"
    start_urls = ["https://books.toscrape.com/"]

    def parse_detail(self, response):
        loader = ProduitLoader(item=ProduitItem(), response=response)
        loader.add_css("titre", "h1::text")
        loader.add_css("prix", "p.price_color::text")
        loader.add_css("prix_numerique", "p.price_color::text")
        loader.add_css("description", "#product_description + p")
        loader.add_value("url", response.url)
        yield loader.load_item()
```

7.3 PIPELINES : TRAITEMENT ET STOCKAGE

```
# pipelines.py
import sqlite3
import json
from itemadapter import ItemAdapter
from datetime import datetime
```

```

class NettoyagePipeline:
    """Valide et nettoie les items avant stockage."""

    def process_item(self, item, spider):
        adapter = ItemAdapter(item)

        # Supprimer les items sans titre ou prix
        if not adapter.get("titre"):
            raise DropItem(f"Item sans titre : {item}")

        # Ajouter la date de scraping
        adapter["date_scrape"] = datetime.now().isoformat()

        # Normaliser les chaînes
        if adapter.get("titre"):
            adapter["titre"] = adapter["titre"].strip()

        return item

class StockageSQLitePipeline:
    """Stocke les items dans une base SQLite."""

    def open_spider(self, spider):
        self.conn = sqlite3.connect("produits.db")
        self.cursor = self.conn.cursor()
        self.cursor.execute("""
            CREATE TABLE IF NOT EXISTS produits (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                titre TEXT,
                prix REAL,
                disponibilite TEXT,
                note TEXT,
                url TEXT UNIQUE,
                date_scrape TEXT
            )
        """)
        self.conn.commit()

    def close_spider(self, spider):
        self.conn.close()

    def process_item(self, item, spider):
        adapter = ItemAdapter(item)
        try:
            self.cursor.execute("""
                INSERT OR REPLACE INTO produits
                (titre, prix, disponibilite, note, url, date_scrape)
                VALUES (?, ?, ?, ?, ?, ?)
            """, (
                adapter.get("titre"),
                adapter.get("prix_numerique"),
                adapter.get("disponibilite"),
                adapter.get("note"),
                adapter.get("url"),
                adapter.get("date_scrape"),
            ))
            self.conn.commit()
        except Exception as e:
            spider.logger.error(f"Erreur stockage : {e}")
        return item

```

PARTIE VIII – GESTION DES URLS : URLLIB, W3LIB, TLDEXTRACT, HYPERLINK

8.1 URLLIB : MANIPULATION D'URLS (STDLIB)

```

from urllib.parse import (
    urlparse, urljoin, urlencode, quote, unquote,
    parse_qs, parse_qsl, urlunparse
)

```

```

# Analyser une URL
url = "https://example.com/produits?page=2&q=python#section1"
parsed = urlparse(url)
print(parsed.scheme)    # https
print(parsed.netloc)    # example.com
print(parsed.path)      # /produits
print(parsed.query)     # page=2&q=python
print(parsed.fragment)  # section1

# Extraire les paramètres GET
params = parse_qs(parsed.query)
print(params)           # {'page': ['2'], 'q': ['python']}

# Construire une URL depuis ses composantes
from urllib.parse import urlencode
base = "https://example.com/search"
params = urlencode({"q": "web scraping", "page": 3, "lang": "fr"})
url_complete = f"{base}?{params}"
print(url_complete)     # https://example.com/search?q=web+scraping&page=3&lang=fr

# Résoudre une URL relative par rapport à une base
base = "https://example.com/produits/categorie/"
relative = "../autre-categorie/item.html"
absolue = urljoin(base, relative)
print(absolue)          # https://example.com/produits/autre-categorie/item.html

# Encoder des caractères spéciaux
print(quote("nom avec espaces & caractères"))
print(unquote("nom%20avec%20espaces"))

```

8.2 W3LIB : UTILITAIRES WEB AVANCÉS

w3lib est la bibliothèque d'utilitaires web de Scrapy. Elle propose des fonctions de nettoyage très utiles en post-traitement.

```

from w3lib.html import (
    remove_tags,
    remove_tags_with_content,
    replace_entities,
    strip_html5_whitespace,
    get_base_url,
    get_meta_refresh,
)
from w3lib.url import (
    safe_url_string,
    canonicalize_url,
    url_query_cleaner,
    add_or_replace_parameter,
)

# Supprimer tous les tags HTML
html = "<p>Bonjour <strong>monde</strong> !</p>"
print(remove_tags(html))           # "Bonjour monde !"

# Supprimer des tags et leur contenu
html = "<p>Texte <script>alert(1)</script> propre</p>"
print(remove_tags_with_content(html, which_ones=("script", "style")))

# Décoder les entités HTML
print(replace_entities("Prix : &euro;29.90 &amp; livraison"))
# "Prix : €29.90 & livraison"

# Normaliser une URL (canonicalize)
url = "https://example.com/page?b=2&a=1&utm_source=google"
print(canonicalize_url(url))
# https://example.com/page?a=1&b=2&utm_source=google

# Nettoyer les paramètres de tracking
print(url_query_cleaner(url, parameterlist=["utm_source", "utm_medium"]))
# https://example.com/page?b=2&a=1

# Ajouter ou remplacer un paramètre

```

```
print(add_or_replace_parameter(url, "page", "2"))
```

8.3 TLDEXTRACT : DÉCOMPOSER LES DOMAINES

```
import tldextract

urls = [
    "https://www.news.bbc.co.uk/article/123",
    "https://subdomain.example.com/path",
    "https://api.github.io/users",
]

for url in urls:
    ext = tldextract.extract(url)
    print(f"sous-domaine: {ext.subdomain}") # www.news, subdomain, api
    print(f"domaine: {ext.domain}")        # bbc, example, github
    print(f"TLD: {ext.suffix}")            # co.uk, com, io
    print(f"domaine complet: {ext.registered_domain}") # bbc.co.uk

# Utilisation pratique : filtrer les URLs par domaine
def meme_domaine(url1, url2):
    return (tldextract.extract(url1).registered_domain ==
            tldextract.extract(url2).registered_domain)
```

PARTIE IX – ENCODAGE ET NORMALISATION

9.1 CHARSET_NORMALIZER : DÉTECTER L'ENCODAGE

Quand une page ne déclare pas son encodage ou le déclare incorrectement, `charset_normalizer` le détecte automatiquement.

```
from charset_normalizer import from_bytes, from_path

# Détecter l'encodage d'un contenu bytes
contenu = requests.get(url).content # bytes bruts
result = from_bytes(contenu).best()

if result:
    print(f"Encodage détecté : {result.encoding}")
    print(f"Confiance : {result.chaos:.2%}") # 0% = très confiant
    texte = str(result)

# Directement depuis un fichier
result = from_path("page_inconnue.html").best()
print(result.encoding)

# Avec requests : forcer l'encodage détecté
response = requests.get(url)
if response.encoding == "ISO-8859-1":
    # Souvent faux par défaut, on détecte le vrai
    detected = from_bytes(response.content).best()
    if detected:
        response.encoding = detected.encoding
print(response.text)
```

9.2 WEBENCODINGS : ENCODAGES SELON LA SPEC HTML5

```
import webencodings

# Normaliser un nom d'encodage selon la spec HTML5
print(webencodings.lookup("latin1")) # <Encoding latin-1>
print(webencodings.lookup("win-1252")) # <Encoding windows-1252>

# Décoder des bytes avec gestion d'erreurs HTML5
label = "windows-1252"
encoding = webencodings.lookup(label)
texte, had_errors = webencodings.decode(b"Caf\xe9", encoding)
```



```
print(texte) # "Café"
```

PARTIE X – SCRAPING ASYNCHRONE : AIOHTTP + ASYNCIO

10.1 AIOHTTP : CLIENT HTTP ASYNCHRONE HAUTE PERFORMANCE

aiohttp est la bibliothèque HTTP asynchrone native pour Python. Parfait pour les scrapers qui doivent traiter des centaines d'URLs en parallèle avec un contrôle fin des connexions.

```
import aiohttp
import asyncio
from bs4 import BeautifulSoup

async def fetch(session, url):
    """Récupère une page de manière asynchrone."""
    try:
        async with session.get(url, timeout=aiohttp.ClientTimeout(total=10)) as r:
            if r.status == 200:
                return await r.text()
            return None
    except Exception as e:
        print(f"Erreur {url}: {e}")
        return None

async def scraper_massif(urls, max_concurrent=10):
    """Scrape de nombreuses URLs en parallèle avec limite de concurrence."""
    semaphore = asyncio.Semaphore(max_concurrent)
    resultats = {}

    headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"}
    connector = aiohttp.TCPConnector(
        limit=max_concurrent,
        ssl=False # désactiver si problèmes SSL
    )

    async def fetch_with_sem(session, url):
        async with semaphore:
            await asyncio.sleep(0.2) # délai poli
            html = await fetch(session, url)
            if html:
                soup = BeautifulSoup(html, "lxml")
                titre = soup.find("h1")
                resultats[url] = titre.text if titre else ""

    async with aiohttp.ClientSession(
        headers=headers, connector=connector
    ) as session:
        taches = [fetch_with_sem(session, url) for url in urls]
        await asyncio.gather(*taches)

    return resultats

# Exécution
urls = [f"https://quotes.toscrape.com/page/{i}/" for i in range(1, 11)]
resultats = asyncio.run(scraper_massif(urls))
print(f"{len(resultats)} pages récupérées")
```

PARTIE XI – PROXIES ET ANONYMAT

11.1 PROXIES AVEC REQUESTS

```
import requests

# Proxy HTTP simple
proxies = {
    "http": "http://proxy.exemple.com:8080",
```

```

    "https": "https://proxy.exemple.com:8080"
}
response = requests.get("https://httpbin.org/ip", proxies=proxies)
print(response.json())

# Proxy avec authentification
proxies_auth = {
    "https": "https://user:password@proxy.exemple.com:8080"
}

# Rotation de proxies
import random

LISTE_PROXIES = [
    "http://proxy1.exemple.com:8080",
    "http://proxy2.exemple.com:8080",
    "http://proxy3.exemple.com:8080",
]

def get_random_proxy():
    proxy = random.choice(LISTE_PROXIES)
    return {"http": proxy, "https": proxy}

response = requests.get("https://httpbin.org/ip",
    proxies=get_random_proxy())

```

11.2 PROXY SOCKS AVEC PYSOCKS ET SOCKSHANDLER

```

# Avec requests (via PySocks)
import requests
import socks
from sockshandler import SocksiPyHandler
import urllib.request

# SOCKS5 avec requests
proxies = {
    "http": "socks5://127.0.0.1:9050",    # Tor par défaut sur ce port
    "https": "socks5://127.0.0.1:9050"
}
response = requests.get("https://check.torproject.org/", proxies=proxies)

# SOCKS5 avec urllib (via SocksiPyHandler)
opener = urllib.request.build_opener(
    SocksiPyHandler(socks.SOCKS5, "127.0.0.1", 9050)
)
response = opener.open("https://httpbin.org/ip")
print(response.read())

```

11.3 URLLIB3 : POOLING ET CONFIGURATION AVANCÉE

urllib3 est le transport bas niveau derrière requests.
On l'utilise directement pour du contrôle fin.

```

import urllib3
from urllib3.util.retry import Retry
from urllib3.util.timeout import Timeout

# Pool de connexions
http = urllib3.PoolManager(
    num_pools=10,          # nombre de pools de connexions
    maxsize=10,            # connexions par pool
    timeout=Timeout(connect=5.0, read=10.0),
    retries=Retry(total=3, backoff_factor=0.5),
    headers={"User-Agent": "Mozilla/5.0 ..."}
)

response = http.request("GET", "https://httpbin.org/get")
print(response.status)
print(response.data.decode("utf-8"))

# Proxy HTTP avec urllib3

```

```
proxy = urllib3.ProxyManager("http://proxy.exemple.com:8080")
response = proxy.request("GET", "https://example.com")
```

PARTIE XII – SCRAPING DE SITES DYNAMIQUES : SELENIUM

12.1 SELENIUM : AUTOMATISER UN VRAI NAVIGATEUR

Selenium contrôle un navigateur réel (Chrome, Firefox).
Indispensable pour les sites qui chargent leur contenu en JavaScript.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options
import time

# Configuration en mode headless (sans fenêtre)
options = Options()
options.add_argument("--headless")
options.add_argument("--no-sandbox")
options.add_argument("--disable-dev-shm-usage")
options.add_argument("--disable-blink-features=AutomationControlled")
options.add_argument(
    "user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
    "AppleWebKit/537.36 Chrome/120.0.0.0 Safari/537.36"
)

driver = webdriver.Chrome(options=options)

try:
    driver.get("https://example.com")

    # Attendre qu'un élément soit présent (max 10 secondes)
    wait = WebDriverWait(driver, 10)
    element = wait.until(
        EC.presence_of_element_located((By.CSS_SELECTOR, "div.produit"))
    )

    # Extraire le HTML après rendu JavaScript
    html_rendu = driver.page_source
    soup = BeautifulSoup(html_rendu, "lxml")

    # Interagir avec la page
    champ = driver.find_element(By.ID, "recherche")
    champ.send_keys("python scraping")
    champ.submit()

    # Cliquer sur un bouton
    bouton = driver.find_element(By.CSS_SELECTOR, "button.charger-plus")
    bouton.click()

    # Scroll vers le bas (lazy loading)
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
    time.sleep(2)

finally:
    driver.quit()
```

12.2 SELENIUM – SCRAPER UN SITE AVEC INFINITE SCROLL

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
from bs4 import BeautifulSoup
import time

def scraper_infinite_scroll(url, max_scrolls=10):
    options = Options()
```

```

options.add_argument("--headless")
driver = webdriver.Chrome(options=options)

driver.get(url)
time.sleep(2)

hauteur_precedente = 0
scrolls = 0

while scrolls < max_scrolls:
    # Scroll vers le bas
    driver.execute_script(
        "window.scrollTo(0, document.body.scrollHeight);"
    )
    time.sleep(2) # attendre le chargement

    nouvelle_hauteur = driver.execute_script(
        "return document.body.scrollHeight"
    )

    if nouvelle_hauteur == hauteur_precedente:
        print("Fin du contenu atteinte.")
        break

    hauteur_precedente = nouvelle_hauteur
    scrolls += 1
    print(f"Scroll {scrolls}/{max_scrolls}")

html = driver.page_source
driver.quit()
return BeautifulSoup(html, "lxml")

```

PARTIE XIII – NETTOYAGE ET POST-TRAITEMENT

13.1 BLEACH : NETTOYAGE ET SANITISATION HTML

bleach nettoie le HTML en supprimant les tags et attributs non autorisés.
 Utile pour stocker du HTML partiel sans risque XSS.

```

import bleach

html_sale = """
<p onclick="alert()">Texte <script>mauvais()</script>
<a href="http://example.com" style="color:red">lien</a>
<strong>important</strong></p>
"""

# Autoriser seulement certains tags
tags_ok = ["p", "a", "strong", "em", "ul", "li"]
attrs_ok = {"a": ["href", "title"]}

html_propre = bleach.clean(
    html_sale,
    tags=tags_ok,
    attributes=attrs_ok,
    strip=True # supprimer (vs échapper) les tags non autorisés
)
print(html_propre)
# <p>Texte <a href="http://example.com">lien</a> <strong>important</strong></p>

# Convertir les URLs en liens cliquables
texte = "Visitez http://example.com ou https://python.org"
texte_avec_liens = bleach.linkify(texte)

```

13.2 RE ET REGEX : EXTRACTION PAR EXPRESSIONS RÉGULIÈRES

```

import re
import regex # version avancée : Unicode, groupes atomiques, etc.

```

```

texte = "Prix : 29,90 € | Référence : REF-2024-ABC | Email: contact@ex.com"

# Extraire un prix
prix = re.search(r"(\d+[\.,]\d+)\s*€", texte)
if prix:
    print(float(prix.group(1).replace(",", "."))) # 29.9

# Extraire une référence
ref = re.search(r"REF-\d{4}-[A-Z]+", texte)
print(ref.group()) # REF-2024-ABC

# Extraire des emails
emails = re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", texte)
print(emails) # ['contact@ex.com']

# Nettoyage de texte extrait
def nettoyer_texte(texte):
    texte = re.sub(r"\s+", " ", texte) # multiples espaces → 1
    texte = re.sub(r"[\r\n\t]", " ", texte) # retours à la ligne
    texte = re.sub(r"^\w\s€£$.!?-]", "", texte) # caractères spéciaux
    return texte.strip()

# regex (module avancé) : Unicode, lookahead, etc.
# Extraire des numéros de téléphone internationaux
telephones = regex.findall(
    r"(?:\+\d{1,3}[\s.-])?(?(\d{1,4})?[\s.-]\d{1,4}[\s.-]\d{1,9})",
    "Appelez le +33 1 23 45 67 89 ou le 06.12.34.56.78"
)
print(telephones)

```

PARTIE XIV – STOCKAGE DES DONNÉES SCRAPÉES

14.1 JSON, JSONL, CSV AVEC PANDAS

```

import pandas as pd
import json

# Liste de dicts scrapés → DataFrame
donnees = [
    {"titre": "Livre A", "prix": 12.50, "auteur": "Dupont"},
    {"titre": "Livre B", "prix": 8.99, "auteur": "Martin"},
]
df = pd.DataFrame(donnees)

# Export CSV
df.to_csv("livres.csv", index=False, encoding="utf-8-sig")

# Export JSON (records = liste de dicts)
df.to_json("livres.json", orient="records", force_ascii=False, indent=2)

# Export JSONL (1 objet par ligne – idéal pour gros volumes)
with open("livres.jsonl", "w", encoding="utf-8") as f:
    for item in donnees:
        f.write(json.dumps(item, ensure_ascii=False) + "\n")

# Export Excel
df.to_excel("livres.xlsx", index=False)

# Lecture et analyse
df = pd.read_csv("livres.csv")
print(df.describe())
print(df.groupby("auteur")["prix"].mean())

```

14.2 SQLITE AVEC LE MODULE SQLITE3

```

import sqlite3
import pandas as pd

# Connexion et création de table

```

```

conn = sqlite3.connect("scraping.db")
cursor = conn.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS articles (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        titre TEXT NOT NULL,
        url TEXT UNIQUE,
        prix REAL,
        date_scrape TEXT,
        contenu TEXT
    )
""")
conn.commit()

# Insertion avec protection contre les doublons
def inserer_article(conn, article):
    try:
        conn.execute("""
            INSERT OR IGNORE INTO articles
            (titre, url, prix, date_scrape, contenu)
            VALUES (?, ?, ?, ?, ?)
        """, (
            article["titre"],
            article["url"],
            article.get("prix"),
            article.get("date_scrape"),
            article.get("contenu")
        ))
        conn.commit()
    except sqlite3.Error as e:
        print(f"Erreur DB : {e}")

# Lire avec pandas directement depuis SQLite
df = pd.read_sql("SELECT * FROM articles WHERE prix < 20", conn)
print(df)
conn.close()

```

PARTIE XV – BONNES PRATIQUES, ÉTHIQUE ET ROBUSTESSE

15.1 RESPECTER LES SERVEURS ET LES RÈGLES

[1] LIRE ROBOTS.TXT AVANT DE SCRAPER

Le fichier robots.txt d'un site déclare ce qui peut/ne peut pas être crawlé. Le respecter est une obligation éthique et légale.

```

import urllib.robotparser

rp = urllib.robotparser.RobotFileParser()
rp.set_url("https://example.com/robots.txt")
rp.read()
print(rp.can_fetch("*", "https://example.com/page-publique")) # True
print(rp.can_fetch("*", "https://example.com/admin/"))        # False

```

[2] DÉLAI ENTRE LES REQUÊTES

- Minimum 1 seconde entre chaque requête sur un même domaine
- Utiliser RANDOMIZE_DOWNLOAD_DELAY dans Scrapy
- Respecter le Crawl-delay indiqué dans robots.txt

[3] IDENTIFIER SON BOT

- User-Agent clair : "MonBot/1.0 (contact: email@domaine.fr)"
- Ou simuler un navigateur réel si le site le demande

[4] NE PAS SURCHARGER LES SERVEURS

- Limiter la concurrence (max 4-8 requêtes simultanées par domaine)
- Programmer les crawls aux heures creuses si possible
- Mettre en cache les pages déjà téléchargées

15.2 DÉDUPLICATION ET REPRISE SUR ERREUR

```

import hashlib
import shelve

class ScraperAvecReprise:
    """Scraper qui se souvient des URLs déjà traitées."""

    def __init__(self, fichier_etat="etat_scraper"):
        self.etat = shelve.open(fichier_etat)
        self.urls_traitees = set(self.etat.get("urls_traitees", []))

    def deja_traite(self, url):
        return url in self.urls_traitees

    def marquer_traite(self, url):
        self.urls_traitees.add(url)
        self.etat["urls_traitees"] = list(self.urls_traitees)

    def fermer(self):
        self.etat.close()

# Utilisation
scraper = ScraperAvecReprise()
urls = ["https://example.com/p1", "https://example.com/p2"]

for url in urls:
    if scraper.deja_traite(url):
        print(f"Déjà traité : {url}")
        continue
    # ... scraper la page ...
    scraper.marquer_traite(url)

scraper.fermer()

```

15.3 ROTATION DU USER-AGENT ET HEADERS ALÉATOIRES

```

import random

USER_AGENTS = [
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
    "(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 "
    "(KHTML, like Gecko) Version/17.0 Safari/605.1.15",
    "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0",
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
    "(KHTML, like Gecko) Chrome/119.0.0.0 Safari/537.36 Edg/119.0.0.0",
]

def get_headers_aleatoires():
    return {
        "User-Agent": random.choice(USER_AGENTS),
        "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9",
        "Accept-Language": random.choice(["fr-FR,fr;q=0.9", "en-US,en;q=0.8"]),
        "Accept-Encoding": "gzip, deflate, br",
        "Connection": "keep-alive",
        "Cache-Control": "no-cache",
    }

```

PARTIE XVI – PROJETS COMPLETS DE A À Z

16.1 PROJET 1 : VEILLE TARIFAIRE E-COMMERCE

Objectif : surveiller l'évolution des prix d'une liste de produits.
 Modules utilisés : requests, bs4/lxml, pandas, sqlite3, re

```

import requests
from bs4 import BeautifulSoup
import sqlite3
import pandas as pd

```

```

from datetime import datetime
import re
import time

# Base de données
conn = sqlite3.connect("prix.db")
conn.execute("""
    CREATE TABLE IF NOT EXISTS prix (
        produit TEXT, prix REAL, date TEXT, url TEXT
    )
""")

PRODUITS = {
    "Python Crash Course": "https://books.toscrape.com/catalogue/"
                           "a-light-in-the-attic_1000/index.html",
}

headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"}

for nom, url in PRODUITS.items():
    r = requests.get(url, headers=headers, timeout=10)
    soup = BeautifulSoup(r.text, "lxml")

    prix_brut = soup.select_one("p.price_color").text
    prix = float(re.sub(r"[^\d.]", "", prix_brut))
    date = datetime.now().isoformat()

    conn.execute("INSERT INTO prix VALUES (?, ?, ?, ?)",
                 (nom, prix, date, url))
    conn.commit()
    print(f"{nom}: {prix}€ ({date})")
    time.sleep(1)

# Analyse des tendances
df = pd.read_sql(
    "SELECT produit, prix, date FROM prix ORDER BY date", conn
)
print(df.pivot_table(values="prix", index="date", columns="produit"))
conn.close()

```

16.2 PROJET 2 : SCRAPER D'ACTUALITÉS MULTI-SOURCES

Objectif : collecter les titres et résumés de plusieurs sites d'actualités.
 Modules utilisés : httpx, asyncio, bs4, pandas, tqdm

```

import httpx
import asyncio
from bs4 import BeautifulSoup
import pandas as pd
from tqdm import tqdm
from datetime import datetime

SOURCES = {
    "LeMonde": {
        "url": "https://www.lemonde.fr/",
        "selector_titres": "h3.article__title",
        "selector_liens": "a.article__link"
    },
}

async def scraper_source(client, nom, config):
    try:
        r = await client.get(config["url"], timeout=15)
        soup = BeautifulSoup(r.text, "lxml")
        articles = []
        for titre in soup.select(config["selector_titres"])[0:10]:
            articles.append({
                "source": nom,
                "titre": titre.text.strip(),
                "date": datetime.now().strftime("%Y-%m-%d %H:%M"),
            })
        return articles
    except Exception as e:

```



```

        print(f"Erreur {nom}: {e}")
        return []

async def main():
    async with httpx.AsyncClient(follow_redirects=True) as client:
        taches = [
            scraper_source(client, nom, config)
            for nom, config in SOURCES.items()
        ]
        resultats = await asyncio.gather(*taches)

    tous = [art for liste in resultats for art in liste]
    df = pd.DataFrame(tous)
    df.to_csv("actualites.csv", index=False, encoding="utf-8")
    print(f"{len(df)} articles collectés.")
    return df

df = asyncio.run(main())

```

16.3 PROJET 3 : SPIDER SCRAPY COMPLET AVEC PIPELINE

Structure du projet :

```

scrapy_livres/
├── scrapy.cfg
├── scrapy_livres/
│   ├── items.py           → ProduitItem
│   ├── pipelines.py       → Nettoyage + SQLite
│   ├── settings.py        → Configuration
│   └── spiders/
│       └── livres.py       → Spider principal

```

```

# spiders/livres.py – Spider complet
import scrapy
from itemloaders import ItemLoader
from itemloaders.processors import TakeFirst, MapCompose
from scrapy_livres.items import LivreItem
from w3lib.html import remove_tags
import re

def to_float(valeur):
    try:
        return float(re.sub(r"^\d.", "", valeur))
    except:
        return None

class LivreLoader(ItemLoader):
    default_output_processor = TakeFirst()
    description_in = MapCompose(remove_tags, str.strip)
    prix_numerique_in = MapCompose(to_float)

class LivresSpider(scrapy.Spider):
    name = "livres"
    allowed_domains = ["books.toscrape.com"]
    start_urls = ["https://books.toscrape.com/"]

    custom_settings = {
        "DOWNLOAD_DELAY": 0.5,
        "CONCURRENT_REQUESTS": 4,
        "ITEM_PIPELINES": {
            "scrapy_livres.pipelines.NettoyagePipeline": 100,
            "scrapy_livres.pipelines.SQLitePipeline": 200,
        },
        "HTTPCACHE_ENABLED": True,
    }

    def parse(self, response):
        # Liens vers les pages de détail
        for lien in response.css("h3 a::attr(href)":):
            yield response.follow(lien, self.parse_livre)

        # Pagination
        suivant = response.css("li.next a::attr(href)").get()

```

```

if suivant:
    yield response.follow(suivant, self.parse)

def parse_livre(self, response):
    l = LivreLoader(item=LivreItem(), response=response)
    l.add_css("titre", "h1::text")
    l.add_css("prix_brut", "p.price_color::text")
    l.add_css("prix_numerique", "p.price_color::text")
    l.add_css("note", "p.star-rating::attr(class)")
    l.add_css("description", "#product_description ~ p")
    l.add_css("disponibilite", "p.availability::text")
    l.add_value("url", response.url)
    yield l.load_item()

```

```
# Lancer : scrapy crawl livres -o livres_complet.json
```

16.4 RÉFÉRENCE RAPIDE – QUEL MODULE POUR QUEL BESOIN

BESOIN	MODULE(S)
Requête HTTP simple	requests
Session persistante	requests.Session
Retry automatique	urllib3.util.retry.Retry
HTTP/2 + async	httpx.AsyncClient
Proxy SOCKS4/5	PySocks + sockshandler
Parser HTML facile	bs4 + lxml
Sélecteurs CSS dans BS4	soupsieve (inclus dans bs4)
XPath rapide	lxml.html / parsel
CSS + XPath style Scrapy	parsel.Selector
Crawler complet production	scrapy
Items structurés	scrapy.Item + itemadapter
Nettoyage auto des items	itemloaders
Utilitaires URL	urllib.parse + w3lib.url
Extraire TLD / domaine	tldextract
Détecter l'encodage	charset_normalizer
Nettoyer HTML	bleach + w3lib.html
Regex puissante	re + regex
Scraping asynchrone massif	aiohttp + asyncio
Sites JavaScript	selenium
Stocker les résultats	pandas + sqlite3
Barre de progression	tqdm

FIN DU COURS

GCI – GYKHAMINE CONCEPT INVESTIGATION
WEB SCRAPING AVEC PYTHON – ÉDITION COMPLÈTE

"Le scraping n'est pas une intrusion – c'est une lecture automatisée.
La différence entre un bon et un mauvais scraper se mesure au respect
qu'il témoigne envers les serveurs qu'il interroge."
